MyBud Student Assistant

Technical project report

Bikash Kushwaha

Public demo | GitHub repository

MyBud brings five common student tasks into one web application: conversational help, code review, recipe suggestions, travel planning, and expense tracking. This portfolio edition repairs the supplied application, makes its operating modes explicit, and adds persistent expense workflows.

Project outcome

The public demo runs on Vercel native Django in Singapore with Neon PostgreSQL. Local development retains SQLite. Assistant tools provide disclosed sample responses and Python syntax parsing; MYBUD_MODE=demo remains active. Optional Gemini calls require configuration. Expense entries and budgets persist for the current browser session.

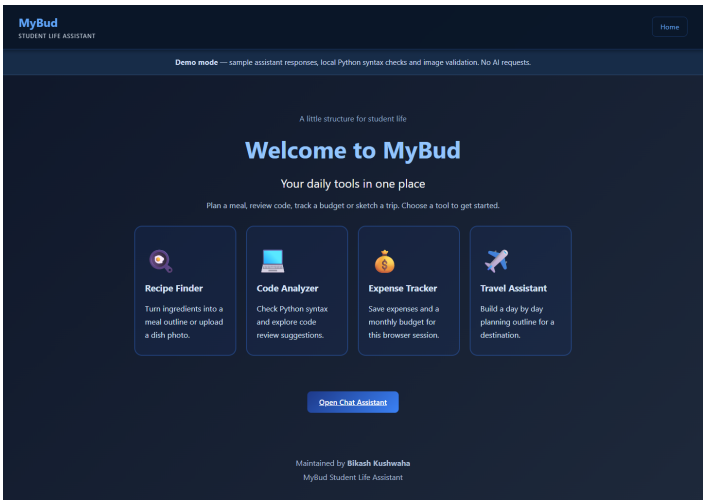


Figure 1. The running dashboard links to the five tools using the preserved dark blue interface.

Area	Scope
Assistant tools	Chat, code review, recipe suggestions with optional image input, and a structured travel itinerary.
Expense tools	Add and delete entries, reset the month, and review NPR spending against a monthly budget.
Evidence boundary	28 tests locally and in GitHub Actions, plus public browser QA. Sample responses do not measure AI quality.

The project is maintained by Bikash Kushwaha and adapted from supplied starter code. The acknowledgement file records that provenance without claiming sole original authorship.

Architecture and request flow

MyBud uses server-rendered Django with JavaScript for tool requests and interface updates. Vercel runs the hosted application in Singapore; Neon PostgreSQL stores its sessions, expenses, and budgets. The browser supplies task input, Django validates it, and a service or database workflow returns the result. Credentials remain on the server.

Layer	Responsibility
Templates and static files	Render the dashboard and tool pages; collect input; present loading, result, and error states.
Routes and views	Map URLs to page handlers or API handlers, validate submitted data, enforce request rules, and scope expense access.
Assistant services	Provide a single configuration boundary for disclosed canned samples or optional live Gemini responses.
Django sessions	Associate the current browser with its expense records and monthly budget.
Persistent models	Neon PostgreSQL on the hosted app, through a pooled DATABASE_URL with TLS. Local development defaults to SQLite.

Assistant request sequence

A user opens a tool page and submits the task fields. The view validates required values and limits before dispatching to the configured provider. The response includes the tool output and sufficient mode information for the interface to identify a sample response. Invalid input and provider failures produce readable errors rather than a server traceback in the page.

Expense request sequence

Django establishes a session key for the browser. Each expense query is restricted to that key and the current month. A validated create writes the record; deleting a record removes it only from the same session. The page reloads or refreshes the returned summary to show the stored monthly totals and budget balance.

Application safeguards

The repairs include CSRF protection for mutating browser requests, validation before provider or database work, and safer display of returned text. Environment-driven settings separate local development from hosting configuration. These measures support the local and hosted demo. Broader production requirements and capacity still need review.

Source references: project/settings.py, project/urls.py, main/urls.py, main/views.py, main/models.py, main/services/, templates/, static/.

Assistant tools and provider modes

The assistant tools share a provider boundary so that the application can be demonstrated offline and configured for a live model without changing the user-facing workflow. The selected mode is disclosed in the interface.

Mode	Behavior and boundary
Sample mode	Default local mode. Returns disclosed demonstration content without model inference or an API key. The code tool also parses Python syntax. This mode demonstrates local behavior without evaluating AI answer quality.
Live Gemini mode	Uses google-genai and the configured model. Requires valid credentials and provider availability. Output quality, latency, quota, and costs depend on that external service.

Tool	User workflow
Chat	Submit a question or study prompt and read the response. Sample mode labels its example content.
Code review	Submit a code snippet and read review suggestions. Demo mode parses Python syntax and returns fixed review tips. This tool does not execute submitted code.
Recipes	Submit ingredients and optionally an image. The request validates the upload before processing and returns recipe suggestions.
Travel	Provide a destination and trip duration. Read the returned day-by-day itinerary.

Image handling

The image endpoint validates JPEG, PNG, or WebP bytes with Pillow. The hosted cap is 3 MB; the local default is 5 MB. Images over 16 million pixels are rejected. Accepted images are converted to RGB, reduced to a maximum 1600 by 1600 bounding box, and re-encoded as JPEG before a possible live request. Demo mode validates the image without claiming food recognition.

Response handling

Input, returned text, and errors are rendered as user-visible content. The interface must not treat arbitrary returned strings as trusted executable markup. Loading and error states let the user understand whether a request is in progress, complete, or rejected.

Evaluation boundary

The verification record confirms the documented local flows. No benchmark score, response accuracy rate, or live-provider performance claim is inferred from sample output. Travel and recipe suggestions should be checked for current facts and individual suitability before use.

Source references: main/views.py, main/services/, .env.example, main/tests.py, docs/VERIFICATION.md.

Expense data and ownership

The expense tracker persists records independently of the assistant provider. Neon PostgreSQL stores the hosted records; SQLite serves local development. Entries remain after reload. Ownership follows the Django browser session rather than an authenticated account.

Data	Purpose
Expense	session_id, month, category, amount, and created_at. Amounts have two decimal places and must be positive.
Budget	session_id, month, amount, and last_set. A unique constraint permits one budget per session and month; zero is allowed.
Monthly summary	Current month in Asia/Kathmandu, NPR total, budget, remaining amount, exceeded flag, entries, and category totals.

Validation and operations

The service converts submitted values to Decimal, rejects non-finite values, negative numbers, boolean values, missing values, values at or above 100000000, and amounts with more than two decimal places. Expense amounts must be greater than zero; the budget may be zero. Valid values are quantized to two decimal places, and summaries use exact database aggregation.

Adding saves the category and amount under the current session and month, assigned using Asia/Kathmandu time. Deleting checks ownership before removing a record. Reset removes that session's expenses and budget for the month. Entries can be replaced through delete and add.

Budget behavior

Setting the budget creates or updates the current month's amount in NPR. The interface compares recorded spending with that amount and calculates the remaining balance. A negative balance means recorded expenses exceed the budget.

The captured example records NPR 800.25 spent against NPR 1,000.00, leaving NPR 199.75. These entries verify the workflow and do not report personal finances.

Session ownership and storage

A separate browser session cannot access another session's records. Losing the cookie can remove access to prior entries. This edition has no account recovery or cross-device synchronization. Hosted persistence uses Neon PostgreSQL on the free tier; SQLite stays local. Database credentials and runtime data remain outside version control. Backup policy, retention, and account ownership remain areas for further development.

Source references: main/models.py, main/migrations/, main/views.py, main/tests.py, .gitignore.

Interface evidence and interaction design

The frontend retains the project's dark blue visual direction and consistent input and result areas. These screenshots capture local demo evidence. The hosted app uses the same templates, with a 3 MB upload cap compared with the local 5 MB default.

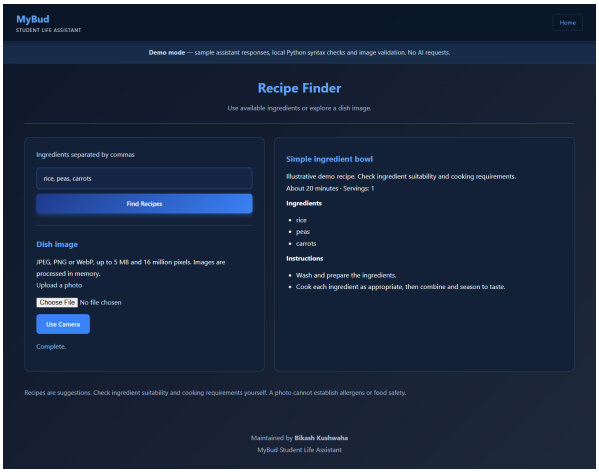


Figure 2. The recipe workflow accepts ingredients and an optional image. The displayed output identifies the active sample mode.

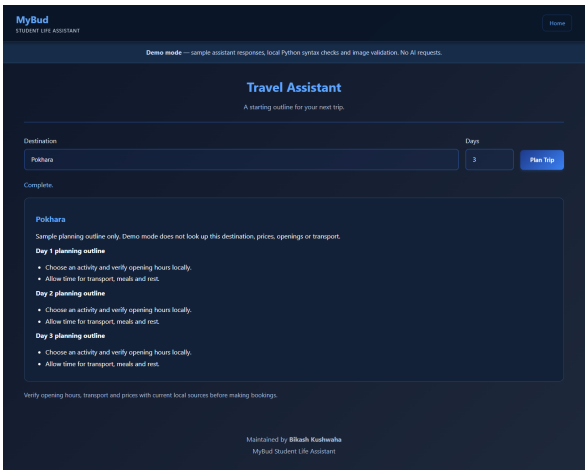


Figure 3. The travel tool organizes its result as an itinerary after destination and duration input.

Readable state changes

A submission communicates that work is in progress. A successful request presents a visible result; an invalid request presents a readable error. The interface preserves user input long enough to correct a rejected value, and the tool pages retain navigation back to the dashboard.

Expense interaction evidence

The expense workflow was checked with records and a budget, including data persistence after reload and the delete and reset paths. The summary reflects the current month in NPR. Browser-session isolation is verified separately from the visual design.

Source references: docs/screenshots/, templates/, static/, docs/VERIFICATION.md.

Verification and observed results

The local suite passed 28 backend tests, including PostgreSQL configuration and TLS checks. Django system and migration checks passed. GitHub Actions run 39472651723 also passed all 28 tests and Ruff lint and format checks for commit 7d7b1de. Public browser QA then verified the deployed demo.

Area	Recorded evidence
Backend and CI	28 tests plus Ruff lint and format checks passed
Django checks	No system-check issues; no migration drift
Hosted storage	Neon PostgreSQL with TLS; migrations completed
Public browser QA	All tools, uploads, expenses and session isolation
Responsive checks	Six routes at desktop and mobile widths

Behavior covered by the checks

The checks cover routes, displayed identity and mode, API contracts, malformed JSON, input bounds, HTTP methods, CSRF, PostgreSQL configuration, and required database TLS.

Expense checks cover exact Decimal totals, invalid monetary values, budget handling, deletion, reset, and isolation between two browser sessions. Image checks cover valid uploads, malformed content, and a corrupt PNG checksum returning a validation error.

Provider-boundary tests use mocks for live success and failure, missing configuration, invalid schemas, private-error sanitization, and response-envelope overwrite rejection. Submitted code is parsed for Python syntax without execution.

Manual browser verification

Public Edge QA exercised all five tools, a real image upload, expense budget/add/reload/delete, and second-session isolation. Injected chat markup remained text. All six routes passed at 1440 by 1000 and 390 by 844 viewports with no page overflow, console errors or warnings, or failed requests. Camera hardware and authenticated live Gemini calls were not tested.

Hosted verification

The production demo is available at https://mybud-student-assistant.vercel.app. Vercel runs Django in Singapore with Neon PostgreSQL on the free tier. MYBUD_MODE=demo remains active. The hosted image limit is 3 MB. These checks establish the observed application flows, not live model quality or production load capacity.

CI evidence: GitHub Actions run 39472651723.

Source references: docs/VERIFICATION.md, main/tests.py, docs/screenshots/.

Local setup and configuration

With Python 3.12, create an environment, install the verified lockfile, migrate, and collect static assets. Run these Windows PowerShell commands from the project directory. The README contains the full setup sequence.

```
python -m venv .venv
.venv\Scripts\python -m pip install -r requirements-lock.txt
.venv\Scripts\python manage.py migrate
.venv\Scripts\python manage.py collectstatic --noinput
.venv\Scripts\python manage.py runserver
```

Provider configuration

Setting	Purpose
MYBUD_MODE	demo on the hosted app; live optionally enables configured Gemini requests.
GEMINI_API_KEY and GEMINI_MODEL	Required together in live mode. Select a model supported by the configured provider account.
DATABASE_URL	Pooled Neon PostgreSQL URL for the hosted runtime with TLS. Local development defaults to SQLite when this is absent.
Django production	DJANGO_DEBUG=false, an explicit DJANGO_SECRET_KEY, allowed hosts, and trusted HTTPS origins.
Other settings	Proxy trust, MYBUD_MAX_IMAGE_MB, and the local SQLite path are documented in .env.example.

Local settings load from .env. Hosted secrets are configured on Vercel. The runtime uses a pooled Neon DATABASE_URL; migrations completed through a direct connection. Live Gemini remains optional and untested with credentials. Its client has a 30-second timeout and validates response structure.

Official references: Google Gen AI SDK and Django 5.2 deployment checklist.

Verification commands

```
.venv\Scripts\python manage.py check
.venv\Scripts\python manage.py test
.venv\Scripts\python -m ruff check .
.venv\Scripts\python -m ruff format --check .
```

Runtime data

Migrations initialize SQLite locally or PostgreSQL on Vercel. Hosted execution requires a persistent DATABASE_URL. Keep secrets, local database files, virtual environments, and caches outside version control.

Source references: README.md, requirements-lock.txt, .env.example, project/settings.py, .gitignore.

Limitations provenance and future work

Current limitations

The hosted app uses disclosed demo responses and does not evaluate a language model. Live Gemini needs credentials and remains untested with them. Browser sessions provide ownership without accounts or recovery. Neon free-tier resource limits apply; SQLite is only the local default. The live-request rate counter is process-local. No payment processing, booking, code execution, or guaranteed advice is included.

Future development

Priority	Proposed work
Ownership and recovery	Introduce account authentication, explicit data ownership, recovery, and an account data deletion flow.
Live assistant evaluation	Record a reviewed prompt set, provider/model configuration, output checks, and failure handling using live credentials.
Expense portability	Add a reviewed export format, backup policy, and explicit retention controls for the hosted PostgreSQL data.
Operations and usability	Add shared rate controls and observability, evaluate capacity, and test the interface with representative users.

Source acknowledgement

This portfolio edition is maintained by Bikash Kushwaha and is adapted from supplied starter code. Maintenance work includes repaired routes and tool flows, a configurable assistant provider, persistent expense operations, input validation, configuration cleanup, and verification materials. The edition does not claim sole original authorship of all starter assets or implementation.

Project evidence references

Reference	Use in this report
README.md and deployment guide	Public links, setup, operating modes, and hosting configuration.
main/ and project/	Implemented behavior, models, tests, routing, and settings.
docs/VERIFICATION.md	Recorded checks and test outcomes.
docs/screenshots/	Captured evidence from the running application.
Source acknowledgement	SOURCE_ACKNOWLEDGEMENTS.md records the supplied starter-code provenance.